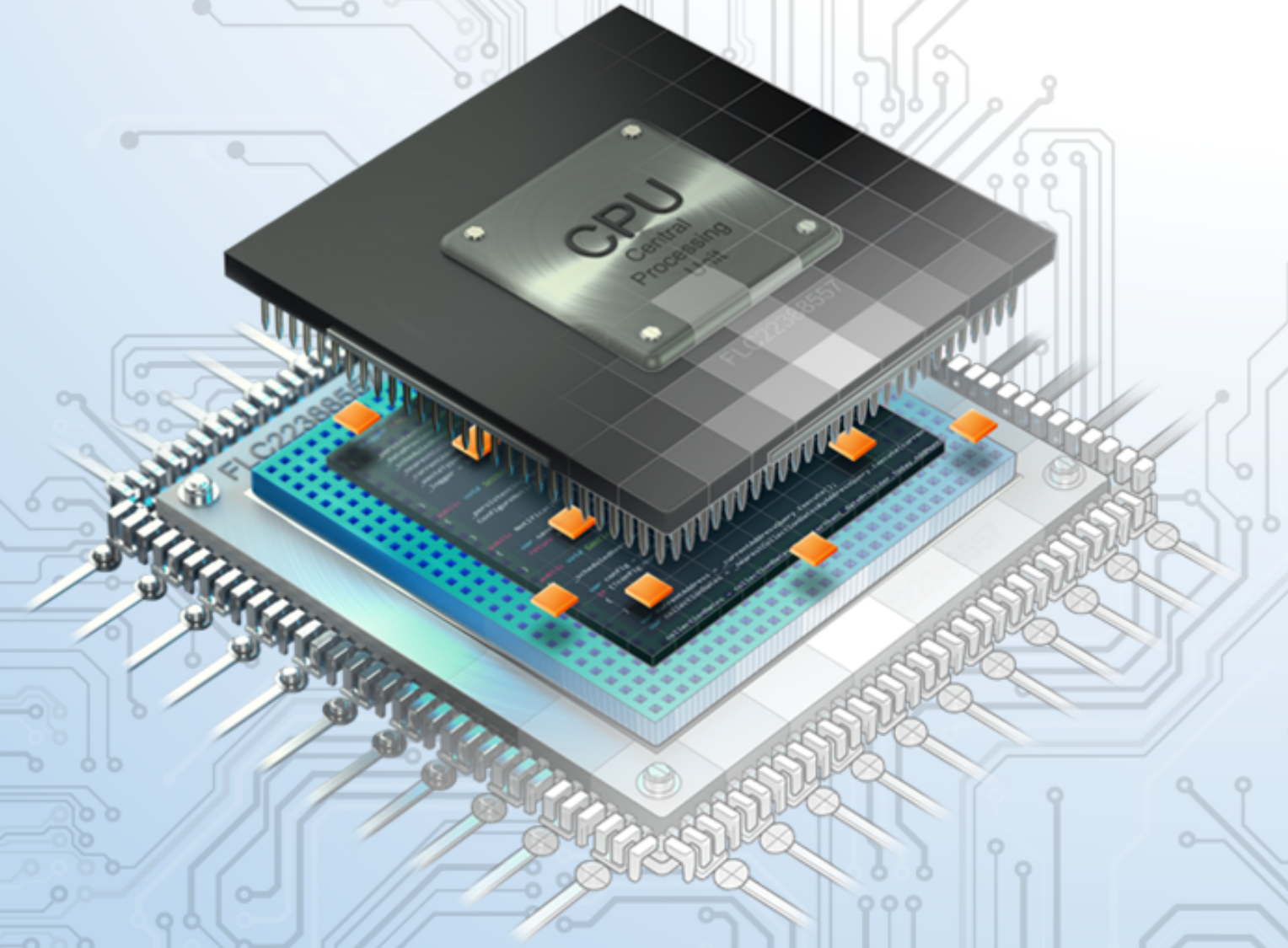
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



Dr. Le Trong Nhan

Mục lục

Chapter 1. Buttons/Switches **7**

1 Objectives 8

2 Introduction 8

3 Basic techniques for reading from port pins 10

3.1 The need for pull-up resistors 10

3.2 Dealing with switch bounces 10

4 Reading switch input (basic code) using STM32 15

4.1 Input Output Processing Patterns 15

4.2 Setting up 16

4.2.1 Create a project 16

4.2.2 Create a file C source file and header file for input reading 16

4.3 Code For Read Port Pin and Debouncing 18

4.3.1 The code in the input_reading.c file 18

4.3.2 The code in the input_reading.h file 19

4.3.3 The code in the timer.c file 19

4.4 Button State Processing 20

4.4.1 Finite State Machine 20

4.4.2 The code for the FSM in the input_processing.c file 21

4.4.3 The code in the input_processing.h 21

4.4.4 The code in the main.c file 22

5 Exercises and Report 23

5.1 Specifications 23

5.2 Exercise 1: Sketch an FSM 24

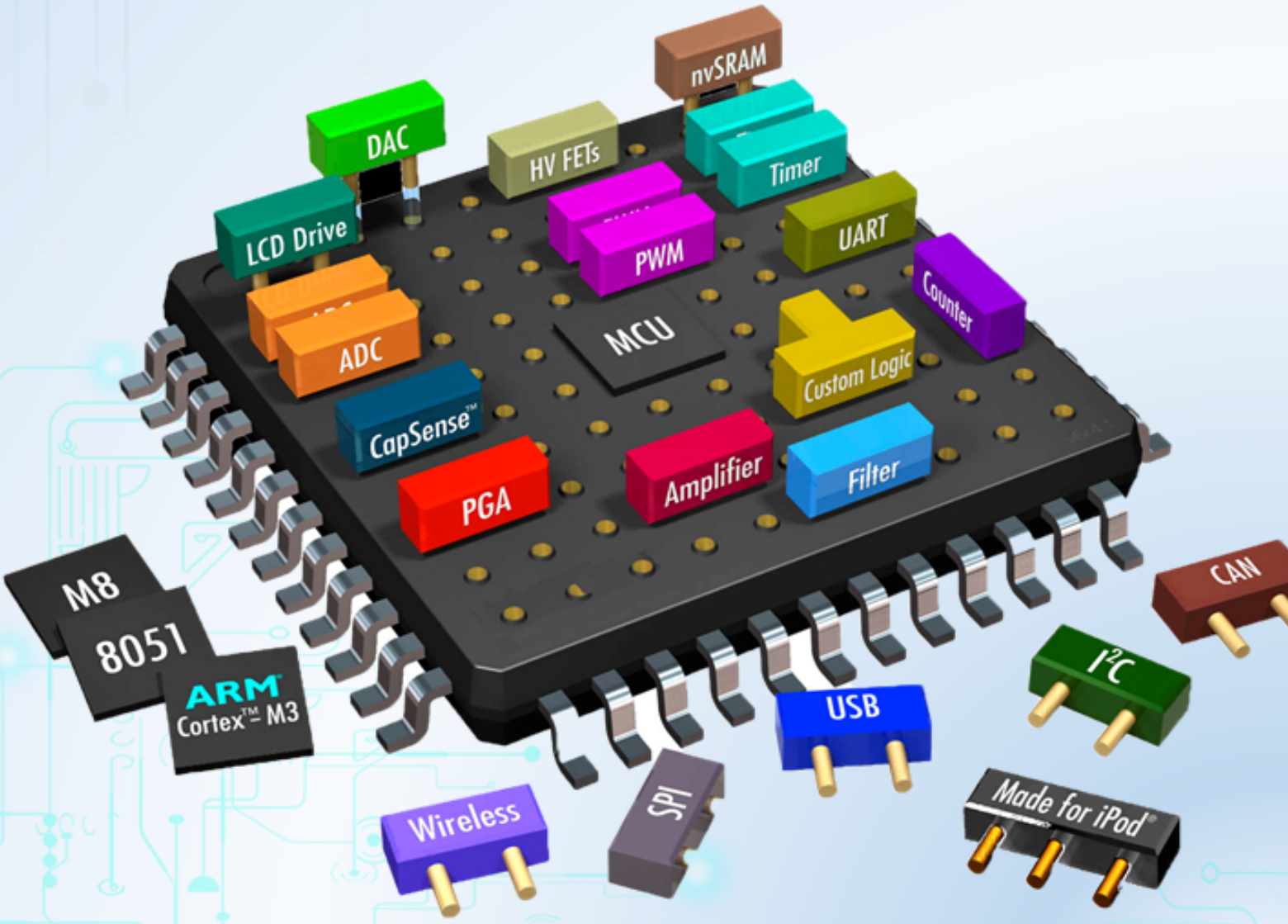
5.3 Exercise 2: Proteus Schematic 24

5.4 Exercise 3: Create STM32 Project 25

5.5 Exercise 4: Modify Timer Parameters 26
5.6 Exercise 5: Adding code for button debouncing 27
5.7 Exercise 6: Adding code for displaying modes 28
5.8 Exercise 7: Increasing and setting the red duration 30
5.9 Exercise 8: Increasing and setting the amber duration 31
5.10 Exercise 9: Increasing and setting the green duration 31
5.11 Exercise 10: Integrated project and verification 32

CHƯƠNG 1

Buttons/Switches



1 Objectives

In this lab, you will

- Learn how to add new C source files and C header files in an STM32 project,
- Learn how to read digital inputs and display values to LEDs using a timer interrupt of a microcontroller (MCU).
- Learn how to debounce when reading a button.
- Learn how to create an FSM and implement an FSM in an MCU.

2 Introduction

Embedded systems usually use buttons (or keys, or switches, or any form of mechanical contacts) as part of their user interface. This general rule applies from the most basic remote-control system for opening a garage door, right up to the most sophisticated aircraft autopilot system. Whatever the system you create, you need to be able to create a reliable button interface.

A button is generally hooked up to an MCU so as to generate a certain logic level when pushed or closed or “active" and the opposite logic level when unpushed or open or “inactive." The active logic level can be either ‘0’ or ‘1’, but for reasons both historical and electrical, an active level of ’0’ is more common.

We can use a button if we want to perform operations such as:

- Drive a motor while a switch is pressed.
- Switch on a light while a switch is pressed.
- Activate a pump while a switch is pressed.

These operations could be implemented using an electrical button without using an MCU; however, use of an MCU may well be appropriate if we require more complex behaviours. For example:

- Drive a motor while a switch is pressed.

  Condition: If the safety guard is not in place, don’t turn the motor. Instead sound a buzzer for 2 seconds.

- Switch on a light while a switch is pressed.

  Condition: To save power, ignore requests to turn on the light during daylight hours.

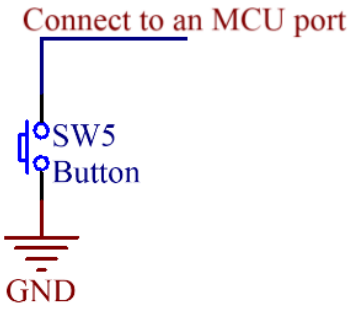
- Activate a pump while a switch is pressed

  Condition: If the main water reservoir is below 300 litres, do not start the main pump: instead, start the reserve pump and draw the water from the emergency tank.

In this lab, we consider how you read inputs from mechanical buttons in your embedded application using an MCU.

3 Basic techniques for reading from port pins

3.1 The need for pull-up resistors



Hình 1.1: *Connecting a button to an MCU*

Figure 1.1 shows a way to connect a button to an MCU. This hardware operates as follows:

- When the switch is open, it has no impact on the port pin. An internal resistor on the port “pulls up" the pin to the supply voltage of the MCU (typically 3.3V for STM32F103). If we read the pin, we will see the value ‘1’.
- When the switch is closed (pressed), the pin voltage will be 0V. If we read the pin, we will see the value ‘0’.

However, if the MCU does not have a pull-up resistor inside, when the button is pressed, the read value will be ‘0’, but even we release the button, the read value is still ‘0’ as shown in Figure 1.2.

So a reliable way to connect a button/switch to an MCU is that we explicitly use an external pull-up resistor as shown in Figure 1.3.

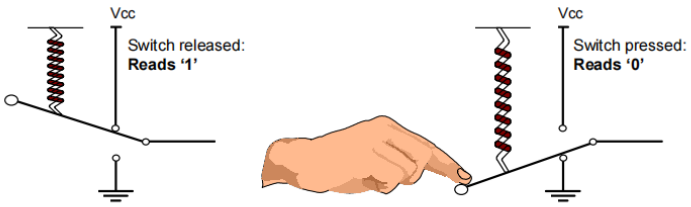
3.2 Dealing with switch bounces

In practice, all mechanical switch contacts bounce (that is, turn on and off, repeatedly, for short period of time) after the switch is closed or opened as shown in Figure 1.4.

Every system that uses any kind of mechanical switch must deal with the issue of debouncing. The key task is to make sure that one mechanical switch or button action is only read as one action by the MCU, even though the MCU will typically be fast enough to detect the unwanted switch bounces and treat them as separate events. Bouncing can be eliminated by special ICs or by RC circuitry, but in most cases debouncing is done in software because software is “free".

As far as the MCU concerns, each “bounce" is equivalent to one press and release of an “ideal” switch. Without appropriate software design, this can give several problems:

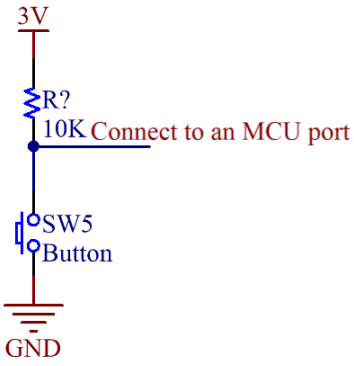
With pull-ups:



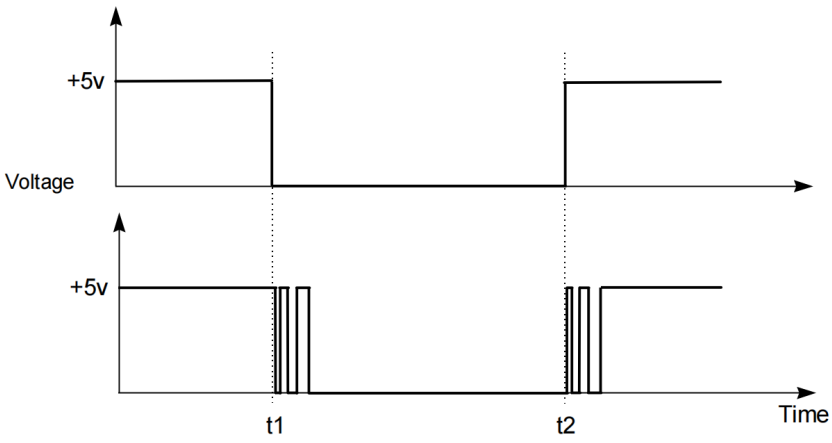
Without pull-ups:



Hình 1.2: The need of pull up resistors



Hình 1.3: A reliable way to connect a button to an MCU



Hình 1.4: Switch bounces

- Rather than reading 'A' from a keypad, we may read 'AAAAA'
- Counting the number of times that a switch is pressed becomes extremely difficult
- If a switch is depressed once, and then released some time later, the 'bounce' may make it appear as if the switch has been pressed again (at the time of release).

The key to debouncing is to establish a minimum criterion for a valid button push, one that can be implemented in software. This criterion must involve differences in time - two button presses in 20ms must be treated as one button event, while two button presses in 2 seconds must be treated as two button events. So what are the relevant times we need to consider? They are these:

- Bounce time: most buttons seem to stop bouncing within 10ms
- Button press time: the shortest time a user can press and release a button seems to be between 50 and 100ms
- Response time: a user notices if the system response is 100ms after the button press, but not if it is 50ms after

Combining all of these times, we can set a few goals

- Ignore all bouncing within 10ms
- Provide a response within 50ms of detecting a button push (or release)
- Be able to detect a 50ms push and a 50ms release

The simplest debouncing method is to examine the keys (or buttons or switches) every N milliseconds, where N > 10ms (our specified button bounce upper limit) and N <= 50ms (our specified response time). We then have three possible outcomes every time we read a button:

- We read the button in the solid '0' state
- We read the button in the solid '1' state
- We read the button while it is bouncing (so we will get either a '0' or a '1')

Outcomes 1 and 2 pose no problems, as they are what we would always like to happen. Outcome 3 also poses no problem because during a bounce either state is acceptable. If we have just pressed an active-low button and we read a '1' as it bounces, the next time through we are guaranteed to read a '0' (remember, the next time through all bouncing will have ceased), so we will just detect the button push a bit later. Otherwise, if we read a '0' as the button bounces, it will still be '0' the next time after all bouncing has stopped, so we are just detecting the button push a bit earlier. The same applies to releasing a button. Reading a single bounce (with all bouncing over by the time of the next read) will never give us an invalid button state. It's only reading multiple bounces (multiple reads while bouncing is

occurring) that can give invalid button states such as repeated push signals from one physical push.

So if we guarantee that all bouncing is done by the time we next read the button, we're good. Well, almost good, if we're lucky...

MCUs often live among high-energy beasts, and often control the beasts. High energy devices make electrical noise, sometimes great amounts of electrical noise. This noise can, at the worst possible moment, get into your delicate button-and-high-value-pullup circuit and act like a real button push. Oops, missile launched, sorry!

If the noise is too intense we cannot filter it out using only software, but will need hardware of some sort (or even a redesign). But if the noise is only occasional, we can filter it out in software without too much bother. The trick is that instead of regarding a single button 'make' or 'break' as valid, we insist on N contiguous makes or breaks to mark a valid button event. N will be a factor of your button scanning rate and the amount of filtering you want to add. Bigger N gives more filtering. The simplest filter (but still a big improvement over no filtering) is just an N of 2, which means compare the current button state with the last button state, and only if both are the same is the output valid.

Note that now we have not two but three button states: active (or pressed), inactive (or released), and indeterminate or invalid (in the middle of filtering, not yet filtered). In most cases we can treat the invalid state the same as the inactive state, since we care in most cases only about when we go active (from whatever state) and when we cease being active (to inactive or invalid). With that simplification we can look at simple **N = 2** filtering reading a button wired to STM32 MCU:

```
1  void button_reading(void){
2      static unsigned char last_button;
3      unsigned char raw_button;
4      unsigned char filtered_button;
5      last_button = raw_button;
6      raw_button = HAL_GPIO_ReadPin(BUTTON_1_GPIO_Port,
       BUTTON_1_Pin);
7      if(last_button == raw_button){
8          filtered_button = raw_button;
9      }
10 }
```

Program 1.1: Read port pin and deboucing

The function button_reading() must be called no more often than our debounce time (10ms).

To expand to greater filtering (larger N), keep in mind that the filtering technique essentially involves reading the current button state and then either counting or reseting the counter. We count if the current button state is the same as the last button state, and if our count reaches N we then report a valid new button state. We reset the counter if the current button state is different than the last button state, and we then save the current button state as the new button state to compare against the next time. Also note that the larger our value of N the more often our filtering routine must be called, so that we get a filtered response within our

specified 50ms deadline. So for example with an N of 8 we should be calling our filtering routine every 2 - 5ms, giving a response time of 16 - 40ms (>10ms and <50ms).

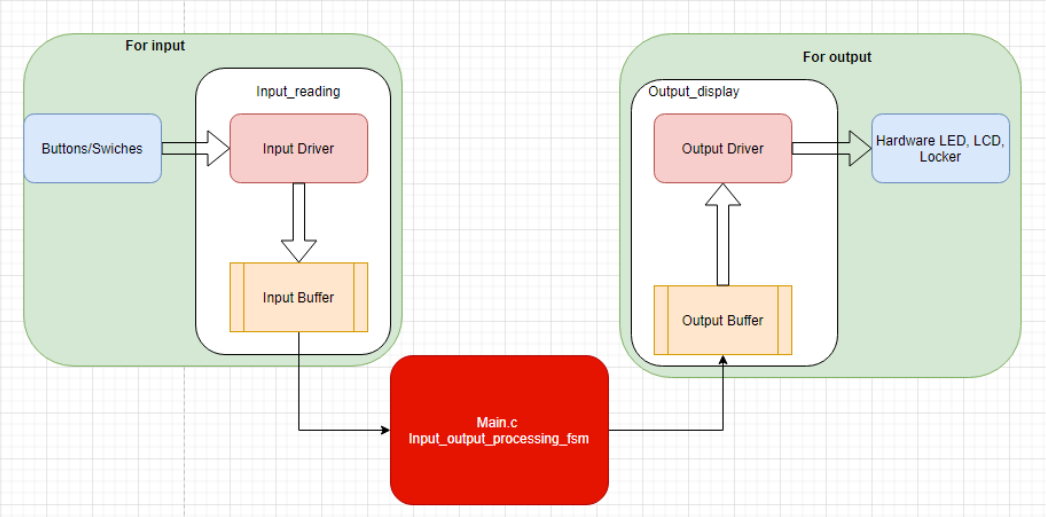
4 Reading switch input (basic code) using STM32

To demonstrate the use of buttons/switches in STM32, we use an example which requires to write a program that

- Has a timer which has an interrupt in every 10 milliseconds.
- Reads values of button PB0 every 10 milliseconds.
- Increases the value of LEDs connected to PORTA by one unit when the button PB0 is pressed.
- Increases the value of PORTA automatically in every 0.5 second, if the button PB0 is pressed in more than 1 second.

4.1 Input Output Processing Patterns

For both input and output processing, we have a similar pattern to work with. Normally, we have a module named driver which works directly to the hardware. We also have a buffer to store temporarily values. In the case of input processing, the driver will store the value of the hardware status to the buffer for further processing. In the case of output processing, the driver uses the buffer data to output to the hardware.



Hình 1.5: Input Output Processing Patterns

Figure 1.5 shows an input-reading module that processes buttons and stores the results in a buffer. An input/output processing module then updates the output buffer. The output driver transfers these values to the hardware.

4.2 Setting up

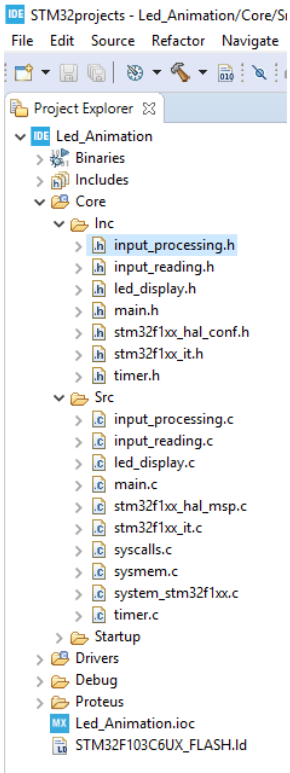
4.2.1 Create a project

Please follow the instruction in Labs 1 and 2 to create a project that includes:

- PB0 as an input port pin,
- PA0-PA7 as output port pins, and
- Timer 2 10ms interrupt

4.2.2 Create a file C source file and header file for input reading

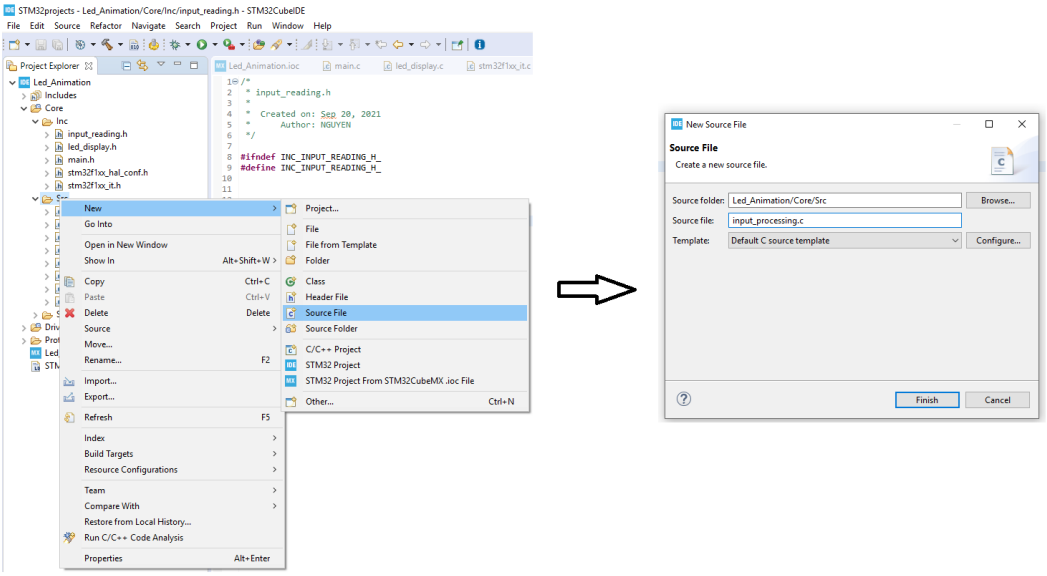
We are expected to have files for button processing and led display as shown in Figure 1.6.



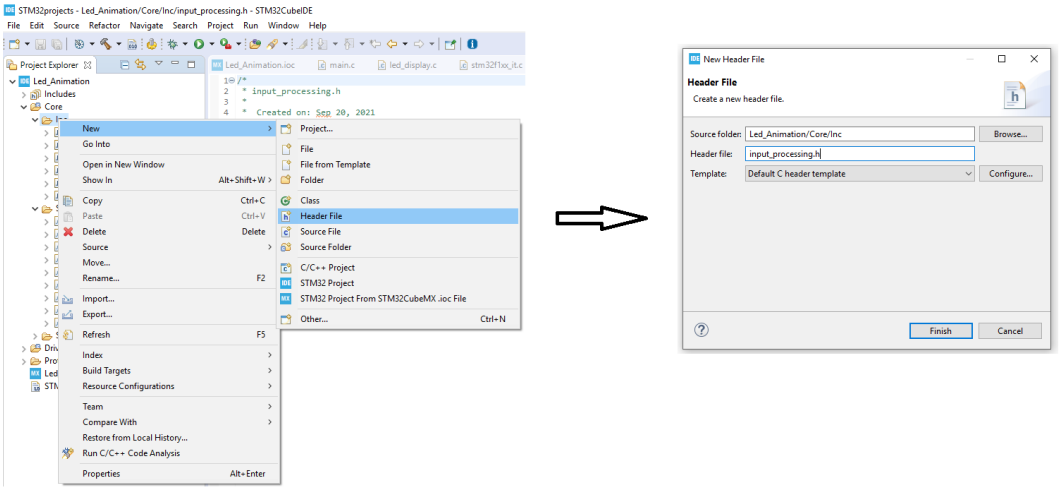
Hình 1.6: *File Organization*

Steps 1 (Figure 1.7): Right click to the folder **Src**, select **New**, then select **Source File**. There will be a pop-up. Please type the file name, then click **Finish**.

Step 2 (Figure 1.8): Do the same for the C header file in the folder **Inc**.



Hình 1.7: Step 1: Create a C source file for input reading



Hình 1.8: Step 2: Create a C header file for input processing

4.3 Code For Read Port Pin and Debouncing

4.3.1 The code in the input_reading.c file

```
#include "main.h"
//we aim to work with more than one buttons
#define NO_OF_BUTTONS                 1
//timer interrupt duration is 10ms, so to pass 1 second,
//we need to jump to the interrupt service routine 100 time
#define DURATION_FOR_AUTO_INCREASING     100
#define BUTTON_IS_PRESSED                  GPIO_PIN_RESET
#define BUTTON_IS_RELEASED                 GPIO_PIN_SET
//the buffer that the final result is stored after
//debouncing
static GPIO_PinState buttonBuffer[NO_OF_BUTTONS];
//we define two buffers for debouncing
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS];
//we define a flag for a button pressed more than 1 second.
static uint8_t flagForButtonPress1s[NO_OF_BUTTONS];
//we define counter for automatically increasing the value
//after the button is pressed more than 1 second.
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];
void button_reading(void){
  for(char i = 0; i < NO_OF_BUTTONS; i ++){
    debounceButtonBuffer2[i] =debounceButtonBuffer1[i];
    debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(
   BUTTON_1_GPIO_Port, BUTTON_1_Pin);
    if(debounceButtonBuffer1[i] == debounceButtonBuffer2[i
   ])
      buttonBuffer[i] = debounceButtonBuffer1[i];
      if(buttonBuffer[i] == BUTTON_IS_PRESSED){
      //if a button is pressed, we start counting
        if(counterForButtonPress1s[i] <
   DURATION_FOR_AUTO_INCREASING){
          counterForButtonPress1s[i]++;
        } else {
        //the flag is turned on when 1 second has passed
        //since the button is pressed.
          flagForButtonPress1s[i] = 1;
          //todo
        }
      } else {
        counterForButtonPress1s[i] = 0;
        flagForButtonPress1s[i] = 0;
      }
  }
}
```

Program 1.2: Define constants buffers and button_reading function

```c
unsigned char is_button_pressed(uint8_t index){
  if(index >= NO_OF_BUTTONS) return 0;
  return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}
```

Program 1.3: Checking a button is pressed or not

```c
unsigned char is_button_pressed_1s(unsigned char index){
  if(index >= NO_OF_BUTTONS) return 0xff;
  return (flagForButtonPress1s[index] == 1);
}
```

Program 1.4: Checking a button is pressed more than a second or not

4.3.2 The code in the input_reading.h file

```c
#ifndef INC_INPUT_READING_H_
#define INC_INPUT_READING_H_
void button_reading(void);
unsigned char is_button_pressed(unsigned char index);
unsigned char is_button_pressed_1s(unsigned char index);
#endif /* INC_INPUT_READING_H_ */
```

Program 1.5: Prototype in input_reading.h file

4.3.3 The code in the timer.c file

```c
#include "main.h"
#include "input_reading.h"

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if(htim->Instance == TIM2){
    button_reading();
  }
}
```

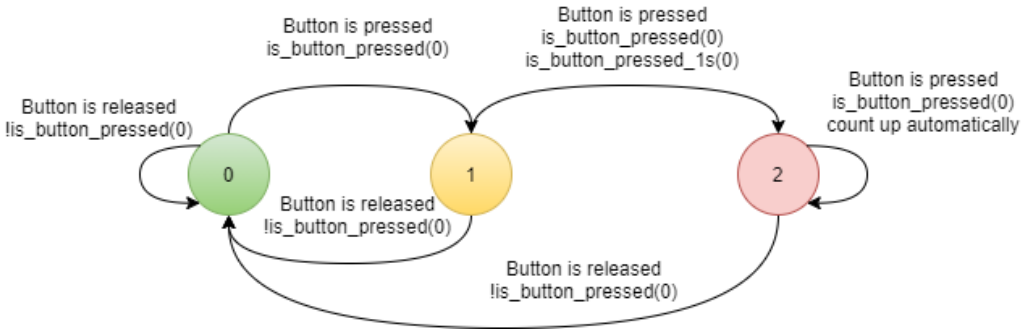
Program 1.6: Timer interrupt callback function

4.4 Button State Processing

4.4.1 Finite State Machine

To solve the example problem, we define 3 states as follows:

- State 0: The button is released or the button is in the initial state.
- State 1: When the button is pressed, the FSM will change to State 1 that is increasing the values of PORTA by one value. If the button is released, the FSM goes back to State 0.
- State 2: while the FSM is in State 1, the button is kept pressing more than 1 second, the state of FSM will change from 1 to 2. In this state, if the button is kept pressing, the value of PORTA will be increased automatically in every 500ms. If the button is released, the FSM goes back to State 0.



Hình 1.9: An FSM for processing a button

4.4.2 The code for the FSM in the input_processing.c file

Please note that *fsm_for_input_processing* function should be called inside the super loop of the main functin.

```
#include "main.h"
#include "input_reading.h"

enum ButtonState{BUTTON_RELEASED, BUTTON_PRESSED,
   BUTTON_PRESSED_MORE_THAN_1_SECOND} ;
enum ButtonState buttonState = BUTTON_RELEASED;
void fsm_for_input_processing(void){
  switch(buttonState){
  case BUTTON_RELEASED:
    if(is_button_pressed(0)){
      buttonState = BUTTON_PRESSED;
      //INCREASE VALUE OF PORT A BY ONE UNIT
    }
    break;
  case BUTTON_PRESSED:
    if(!is_button_pressed(0)){
      buttonState = BUTTON_RELEASED;
    } else {
      if(is_button_pressed_1s(0)){
        buttonState = BUTTON_PRESSED_MORE_THAN_1_SECOND;
      }
    }
    break;
  case BUTTON_PRESSED_MORE_THAN_1_SECOND:
    if(!is_button_pressed(0)){
      buttonState = BUTTON_RELEASED;
    }
    //todo
    break;
  }
}
```

Program 1.7: The code in the input_processing.c file

4.4.3 The code in the input_processing.h

```
#ifndef INC_INPUT_PROCESSING_H_
#define INC_INPUT_PROCESSING_H_

void fsm_for_input_processing(void);

#endif /* INC_INPUT_PROCESSING_H_ */
```

Program 1.8: Code in the input_processing.h file

4.4.4 The code in the main.c file

```
#include "main.h"
#include "input_processing.h"
//don't modify this part
int main(void){
    HAL_Init();
    /* Configure the system clock */
    SystemClock_Config();
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM2_Init();
    while (1)
    {
        //you only need to add the fsm function here
        fsm_for_input_processing();
    }
}
```

Program 1.9: The code in the main.c file

5 Exercises and Report

5.1 Specifications

You are required to build an application of a traffic light in a cross road which includes some features as described below:

- The application has 12 LEDs including 4 red LEDs, 4 amber LEDs, 4 green LEDs.
- The application has 4 seven segment LEDs to display time with 2 for each road. The 2 seven segment LEDs will show time for each color LED corresponding to each road.
- The application has three buttons which are used

  - to select modes,

  - to modify the time for each color led on the fly, and

  - to set the chosen value.

- The application has at least 4 modes which is controlled by the first button. Mode 1 is a normal mode, while modes 2 3 4 are modification modes. You can press the first button to change the mode. Modes will change from 1 to 4 and back to 1 again.

  Mode 1 - Normal mode:

  - The traffic light application is running normally.

  Mode 2 - Modify time duration for the red LEDs: This mode allows you to change the time duration of the red LED in the main road. The expected behaviours of this mode include:

  - All single red LEDs are blinking in 2 Hz.

  - Use two seven-segment LEDs to display the value.

  - Use the other two seven-segment LEDs to display the mode.

  - The second button is used to increase the time duration value for the red LEDs.

  - The value of time duration is in a range of 1 - 99.

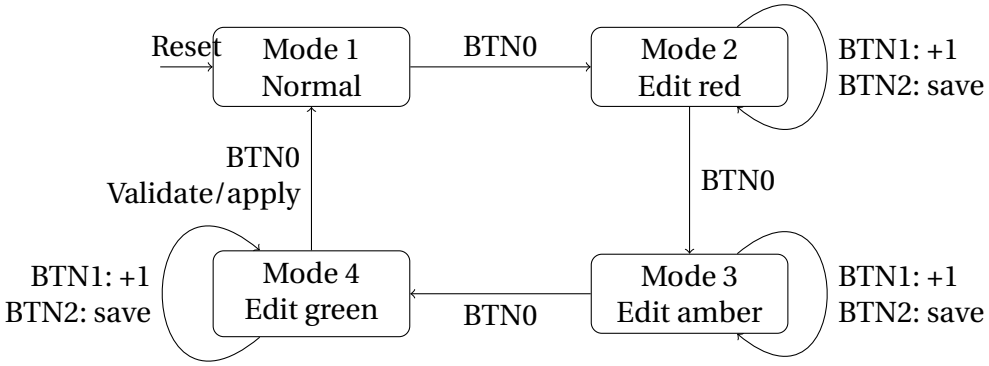
  - The third button is used to set the value.

  Mode 3 - Modify time duration for the amber LEDs: Similar for the red LEDs described above with the amber LEDs.

  Mode 4 - Modify time duration for the green LEDs: Similar for the red LEDs described above with the green LEDs.

5.2 Exercise 1: Sketch an FSM

The application has four modes. BTN0 is the first button in the specification, BTN1 is INCREASE, and BTN2 is SET. An accepted press, not the continuously pressed level, advances the mode. Mode selection has priority over other simultaneous events. Otherwise INCREASE is processed before SET.

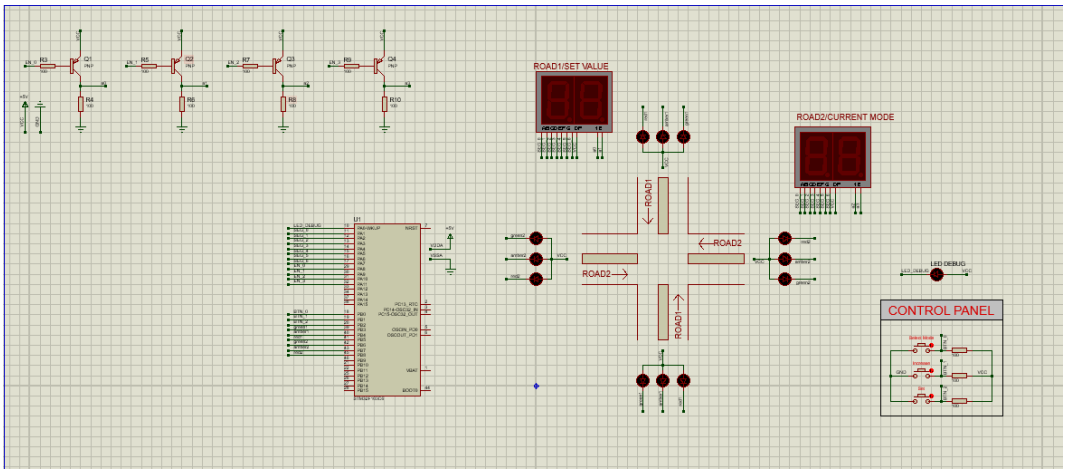


Hình 1.10: Four-mode controller for the traffic-light application.

Normal mode uses the phase sequence red/green, red/amber, green/red, amber/red. The two roads never show green simultaneously in normal operation.

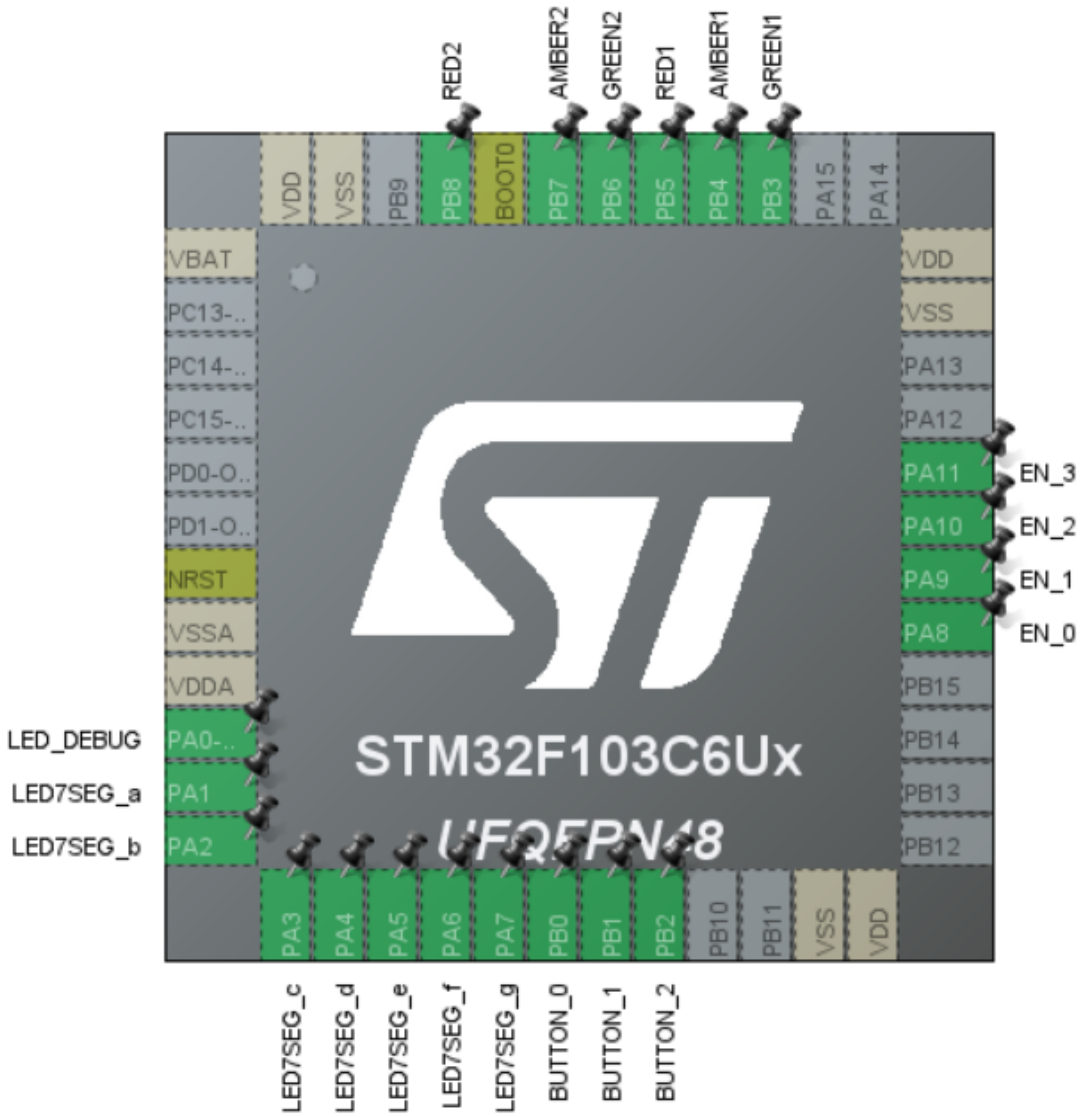
5.3 Exercise 2: Proteus Schematic

The six logical traffic outputs drive paired physical LEDs, giving four red, four amber and four green LEDs. Four logical display digits are multiplexed using a shared segment bus. Each road displays a two-digit number.



Hình 1.11: *Proteus Schematic of Ex2*

5.4 Exercise 3: Create STM32 Project



Hình 1.12: *CubeMX Pinout of Lab 3*

The CubeIDE project is `LAB3/lab_3`, targeting STM32F103C6Ux with an 8 MHz HSI clock and no PLL.

Function	GPIO pins
BTN0 / BTN1 / BTN2	PB0 / PB1 / PB2
Shared segments a–g	PA1–PA7
Digit enables 0–3	PA8–PA11
Road 1 green / amber / red	PB3 / PB4 / PB5
Road 2 green / amber / red	PB6 / PB7 / PB8
Configuration-error indicator	PA0

The inputs use `GPIO_NOPULL` with external pull-up resistors. Outputs are push-pull and start HIGH (OFF). The IOC contains the same initial states. The No Debug

setting frees PB3/PB4 from JTAG. TIM2 uses its internal clock and enabled global interrupt:

$$T_{\mathrm{TIM2}} = \frac{(7999+1)(9+1)}{8\,000\,000} = 10\ \mathrm{ms}.$$

SysTick provides a 1 ms time base for button sampling and display scanning.

5.5 Exercise 4: Modify Timer Parameters

`TIMER_PERIOD` expresses the TIM2 period in milliseconds. Its default is 10. The USER CODE section in `tim.c` sets ARR to `TIMER_PERIOD - 1`, using the 1 kHz prescaled timer clock. Thus the application durations remain 250 ms and 1000 ms when TIM2 changes.

```
#define TIMER_PERIOD 10U
/* With HSI=8 MHz and PSC=7999: */
htim2.Init.Period = TIMER_PERIOD - 1U;
__HAL_TIM_SET_AUTORELOAD(&htim2, TIMER_PERIOD - 1U);
__HAL_TIM_SET_COUNTER(&htim2, 0U);
```

Program 1.10: Single-point TIM2 period configuration

Timers store milliseconds instead of dividing a duration into a truncated 8-bit tick count. Expiry preserves the remainder, preventing accumulating period error. ISR-shared data are volatile; main-loop operations briefly preserve and mask interrupts to update shared state safely.

```
/* Millisecond software timer with phase carry. */
#include "software_timer.h"
#include "tim.h"
typedef struct {
    int32_t remaining_ms;
    uint32_t period_ms, pending;
} SoftwareTimer;
static volatile SoftwareTimer timers[NUMBER_OF_TIMERS];
volatile uint32_t timer_interrupts;
void timer_init(void) {
    __HAL_TIM_CLEAR_FLAG(&htim2, TIM_FLAG_UPDATE);
    if (HAL_TIM_Base_Start_IT(&htim2) != HAL_OK) Error_Handler();
}
void timer_set(int index, int duration_ms) {
    if (index < 0 || index >= (int)NUMBER_OF_TIMERS || duration_ms <= 0) return;
    uint32_t saved = __get_PRIMASK();
    __disable_irq();
    timers[index].period_ms = (uint32_t)duration_ms;
    timers[index].remaining_ms = duration_ms;
    timers[index].pending = 0;
    __set_PRIMASK(saved);
}
/* Called only by TIM2: no truncation of duration_ms / TIMER_PERIOD. */
void timer_run(void) {
    for (unsigned i = 0; i < NUMBER_OF_TIMERS; ++i) {
        if (timers[i].period_ms == 0) continue;
        timers[i].remaining_ms -= (int32_t)TIMER_PERIOD;
        while (timers[i].remaining_ms <= 0) {
            timers[i].remaining_ms += (int32_t)timers[i].period_ms;
            if (timers[i].pending != UINT32_MAX) ++timers[i].pending;
```

```
        }
    }
}
int timer_is_expired(int index) {
    if (index < 0 || index >= (int)NUMBER_OF_TIMERS) return 0;
    uint32_t saved = __get_PRIMASK();
    __disable_irq();
    int expired = timers[index].pending != 0;
    if (expired) --timers[index].pending;
    __set_PRIMASK(saved);
    return expired;
}
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        ++timer_interrupts;
        timer_run();
    }
}
```

Program 1.11: Millisecond timer service driven by TIM2

A 2 Hz blink needs a 500 ms full cycle: 250 ms ON and 250 ms OFF. At 1 ms and 10 ms TIM2 periods these intervals are exactly representable. At 100 ms, edges are quantized: 200/300 ms intervals alternate while the long-term frequency remains 2 Hz. This is a timer-resolution limitation, not exact 250 ms edges. The demonstration firmware uses 10 ms. Button sampling and display scanning use separate SysTick-based deadlines of 10 ms and 5 ms, so a coarse TIM2 experiment does not slow these tasks.

5.6 Exercise 5: Adding code for button debouncing

Each button keeps its own candidate level, accepted level, consecutive-sample count and pending press event. Three equal samples are required before changing the accepted state. A released-to-pressed transition produces one event. Holding the button produces no further press event, and release must also be debounced before another press can be accepted.

```
/* Independent debounced edge events for three buttons. */
#include "button.h"
typedef struct {
    GPIO_TypeDef *port;
    uint16_t pin;
    uint8_t candidate, stable, matches, pressed_event;
    uint32_t held_ms;
} Button;
static Button buttons[NUMBER_OF_BUTTONS] = {
    {BUTTON_0_GPIO_Port, BUTTON_0_Pin, 1, 1, 0, 0, 0},
    {BUTTON_1_GPIO_Port, BUTTON_1_Pin, 1, 1, 0, 0, 0},
    {BUTTON_2_GPIO_Port, BUTTON_2_Pin, 1, 1, 0, 0, 0}
};
void button_init(void) {
    for (unsigned i = 0; i < NUMBER_OF_BUTTONS; ++i) {
        buttons[i].candidate = buttons[i].stable = 1;
        buttons[i].matches = buttons[i].pressed_event = 0;
        buttons[i].held_ms = 0;
    }
```

```
}
/* Call once per BUTTON_SCAN_MS of real time, never replay missed samples. */
void button_scan(void) {
    for (unsigned i = 0; i < NUMBER_OF_BUTTONS; ++i) {
        Button *b = &buttons[i];
        uint8_t raw = (uint8_t)HAL_GPIO_ReadPin(b->port, b->pin);
        if (raw != b->candidate) {
            b->candidate = raw;
            b->matches = 1;
        } else if (b->matches < BUTTON_DEBOUNCE_SAMPLES) {
            ++b->matches;
        }
        if (b->matches == BUTTON_DEBOUNCE_SAMPLES && b->stable != raw) {
            b->stable = raw;
            b->held_ms = 0;
            if (raw == 0) b->pressed_event = 1;
        } else if (b->stable == 0 && raw == 0 && b->held_ms < UINT32_MAX -
    BUTTON_SCAN_MS) {
            b->held_ms += BUTTON_SCAN_MS;
        }
        /* No early return: sample every button on every scan. */
    }
}
uint8_t button_is_pressed(uint8_t index) {
    if (index >= NUMBER_OF_BUTTONS) return 0;
    uint8_t event = buttons[index].pressed_event;
    buttons[index].pressed_event = 0;
    return event;
}
uint8_t button_is_held(uint8_t index, int duration) {
    if (index >= NUMBER_OF_BUTTONS || duration < 0) return 0;
    return buttons[index].stable == 0 && buttons[index].held_ms >= (uint32_t)
    duration;
}
```

Program 1.12: Independent three-sample debounce and one-shot events

All buttons are sampled on every scan. Each button has an independent event flag, so the scan of one button cannot prevent the other buttons from being updated. The application consumes every button event once, including events ignored in normal mode.

5.7 Exercise 6: Adding code for displaying modes

Normal mode displays the remaining time for the two roads. Modification modes display the editable value on road 1's pair and 02, 03 or 04 on road 2's pair. All four physical LEDs of the selected colour blink together; other colours are OFF. Entering a mode loads that colour's previously confirmed value.

```
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
static void edit_outputs(void) {
    lights_off();
    if (blink_on) {
        Color color = (mode == SET_RED) ? RED : ((mode == SET_AMBER) ? AMBER : GREEN
    );
        light_set(0, color);
        light_set(1, color);
```

```
    }
    display_pair(0, edit_value);
    display_pair(1, (uint8_t)mode); /* 02, 03 or 04 */
}
static void enter_edit(AppMode next) {
    mode = next;
    edit_value = saved_value();
    blink_on = 1;
    timer_set(TIMER_LED_BLINKING, 250); /* toggle every 250 ms = 2 Hz */
    edit_outputs();
}
```

Program 1.13: EX6: mode display and selected-colour blinking

All four digit enables are disabled before writing the shared segment bus. One digit is enabled per 5 ms scan; each digit is refreshed every 20 ms, or 50 Hz. The segment encoding is active-low.

```
#include "led7seg.h"
#include "main.h"

#define NUMBER_OF_LED7SEG   4

static uint8_t led7seg_map_of_number[] = {0x40, 0x79, 0x24, 0x30, 0x19, 0x12, 0x02,
    0x78, 0x00, 0x10};
static uint8_t led7seg_map_of_index[] = {0x01, 0x02, 0x04, 0x08};
static uint8_t led7seg_index = 0;
static uint8_t led_7seg[NUMBER_OF_LED7SEG] = {0, 0, 0, 0};

// Hardware function
void _led7seg_display(int num) {
  HAL_GPIO_WritePin(LED7SEG_a_GPIO_Port, LED7SEG_a_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 0)));
  HAL_GPIO_WritePin(LED7SEG_b_GPIO_Port, LED7SEG_b_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 1)));
  HAL_GPIO_WritePin(LED7SEG_c_GPIO_Port, LED7SEG_c_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 2)));
  HAL_GPIO_WritePin(LED7SEG_d_GPIO_Port, LED7SEG_d_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 3)));
  HAL_GPIO_WritePin(LED7SEG_e_GPIO_Port, LED7SEG_e_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 4)));
  HAL_GPIO_WritePin(LED7SEG_f_GPIO_Port, LED7SEG_f_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 5)));
  HAL_GPIO_WritePin(LED7SEG_g_GPIO_Port, LED7SEG_g_Pin, !((~led7seg_map_of_number[
    num]) & (1 << 6)));
}

void _led7seg_en(int num) {
  HAL_GPIO_WritePin(EN_3_GPIO_Port, EN_3_Pin, !((led7seg_map_of_index[num]) & (1 <<
    3)));
  HAL_GPIO_WritePin(EN_2_GPIO_Port, EN_2_Pin, !((led7seg_map_of_index[num]) & (1 <<
    2)));
  HAL_GPIO_WritePin(EN_1_GPIO_Port, EN_1_Pin, !((led7seg_map_of_index[num]) & (1 <<
    1)));
  HAL_GPIO_WritePin(EN_0_GPIO_Port, EN_0_Pin, !((led7seg_map_of_index[num]) & (1 <<
    0)));
}

// APIs
```

```c
void led7seg_scan() {
  HAL_GPIO_WritePin(EN_3_GPIO_Port, EN_3_Pin, 1);
  HAL_GPIO_WritePin(EN_2_GPIO_Port, EN_2_Pin, 1);
  HAL_GPIO_WritePin(EN_1_GPIO_Port, EN_1_Pin, 1);
  HAL_GPIO_WritePin(EN_0_GPIO_Port, EN_0_Pin, 1);

  _led7seg_display(led_7seg[led7seg_index]);
  _led7seg_en(led7seg_index);
  ++led7seg_index;
  if (led7seg_index >= NUMBER_OF_LED7SEG) {
    led7seg_index = 0;
  }
}

void led7seg_set(uint8_t index, uint8_t number) {
  if (index >= NUMBER_OF_LED7SEG || number > 9) return;
  led_7seg[index] = number;
}
```

Program 1.14: EX6: shared-bus four-digit scan

5.8 Exercise 7: Increasing and setting the red duration

BTN1 increases the red edit value by one, wrapping 99 to 1. BTN2 saves it without changing mode. BTN0 discards an unconfirmed edit and enters amber mode.

```c
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
case SET_RED:
        saved.red = edit_value;
        break;
```

Program 1.15: EX7: confirmation of the red draft

The following helper implements the shared edit policy for Exercises 7–9.

```c
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
static void increase_edit(void) {
    edit_value = (edit_value == 99U) ? 1U : edit_value + 1U;
}
/* BTN2 confirms the edited colour. Unsaved changes are discarded on mode change. */
static void save_edit(void) {
    switch (mode) {

    case SET_RED:
        saved.red = edit_value;
        break;


    case SET_AMBER:
        saved.amber = edit_value;
        break;


    case SET_GREEN:
        saved.green = edit_value;
        break;

    default: break;
```

```
    }
}
```

Program 1.16: Shared increment and colour-specific confirmation

Automatic repeat is not enabled because the traffic-light specification only requires one increment for each button press. A debounced press increases the value exactly once; holding BTN1 does not generate additional increments.

5.9 Exercise 8: Increasing and setting the amber duration

Mode 3 displays 03, blinks only amber LEDs, and uses the same one-press/one-step logic. BTN2 updates the confirmed amber draft.

```
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
case SET_AMBER:
        saved.amber = edit_value;
        break;
```

Program 1.17: EX8: confirmation of the amber draft

5.10 Exercise 9: Increasing and setting the green duration

Mode 4 displays 04 and blinks only green LEDs. BTN2 confirms the green draft. The next BTN0 returns to normal and validates the complete confirmed triplet.

```
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
case SET_GREEN:
        saved.green = edit_value;
        break;
```

Program 1.18: EX9: confirmation of the green draft

The implementation enforces the symmetric-road timing relationship $R = Y + G$, with each value in 1–99. This is a design choice, not an explicit additional condition in the handout. Each colour can be confirmed separately; the complete triplet is applied only when leaving mode 4. Invalid triplets retain the previous active timings and light PA0's error LED. Confirmed drafts remain available for correction. A valid later triplet clears the error.

```
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
static uint8_t valid_durations(Durations value) {
    return value.red >= 1U && value.red <= 99U &&
           value.amber >= 1U && value.amber <= 99U &&
           value.green >= 1U && value.green <= 99U &&
           value.red == (uint16_t)value.amber + value.green;
}
static void next_mode(void) {
    switch (mode) {
    case NORMAL: enter_edit(SET_RED); break;
    case SET_RED: enter_edit(SET_AMBER); break;
    case SET_AMBER: enter_edit(SET_GREEN); break;
    case SET_GREEN:
        config_error = !valid_durations(saved);
        if (!config_error) active = saved;
```

```
        /* Invalid drafts stay available for correction. Old active timings remain.
    */
        start_normal();
        break;
    }
}
```

Program 1.19: Draft validation and mode transition

All settings are in RAM; reset restores 5/2/3 seconds. No Flash persistence is claimed or implemented.

5.11 Exercise 10: Integrated project and verification

The final project integrates the mode controller, button events, timer service, four-phase traffic sequencer and multiplexed display driver. Its GPIO mapping is defined consistently by the CubeMX project and the Proteus schematic.

```
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
void app_Select_mode(void) {
    /* Consume each event once, including buttons ignored in the current mode. */
    uint8_t select = button_is_pressed(BUTTON_SELECT_MODE);
    uint8_t increase = button_is_pressed(BUTTON_INCREASE);
    uint8_t confirm = button_is_pressed(BUTTON_SET);
    if (select) {
        next_mode(); /* Mode selection wins if buttons are pressed simultaneously.
    */
    } else if (mode == NORMAL) {
        while (timer_is_expired(TIMER_ONE_SECOND)) traffic_second();
    } else {
        if (increase) increase_edit(); /* One accepted press increases the value by
    one. */
        if (confirm) save_edit();
        while (timer_is_expired(TIMER_LED_BLINKING)) blink_on ^= 1U;
        edit_outputs();
    }
    publish_debug();
}
```

Program 1.20: Integrated event processing

```
/* Main initialization and superloop excerpt. */

  button_init();
  app_Set_timer();
  system_init();
  uint32_t last_button_ms = HAL_GetTick();
  uint32_t last_display_ms = last_button_ms;

  while (1)
  {
    uint32_t now = HAL_GetTick();
    /* Keep input/display responsive even during a 100 ms TIM2 experiment.
     * Never create synthetic debounce samples by replaying missed ticks. */
    if ((uint32_t)(now - last_button_ms) >= BUTTON_SCAN_MS) {
      last_button_ms = now;
      button_scan();
    }
```

```
    if ((uint32_t)(now - last_display_ms) >= 5U) {
      last_display_ms = now;
      led7seg_scan();
    }

    app_Select_mode();

}
```

Program 1.21: Initialization and cooperative main loop

```
/* Extracted from Core/Src/app.c; excerpt, not a standalone program. */
static void traffic_second(void) {
    if (countdown1 > 0) --countdown1;
    if (countdown2 > 0) --countdown2;
    switch (phase) {
    case RED_GREEN:
        if (countdown2 == 0) {
            phase = RED_AMBER;
            countdown2 = active.amber;
        }
        break;
    case RED_AMBER:
        if (countdown2 == 0) {
            phase = GREEN_RED;
            countdown1 = active.green;
            countdown2 = active.red;
        }
        break;
    case GREEN_RED:
        if (countdown1 == 0) {
            phase = AMBER_RED;
            countdown1 = active.amber;
        }
        break;
    case AMBER_RED:
        if (countdown1 == 0) {
            phase = RED_GREEN;
            countdown1 = active.red;
            countdown2 = active.green;
        }
        break;
    }
    traffic_outputs();
}
```

Program 1.22: Normal-mode traffic countdown and phase transitions

Expected demonstration

At reset the displays show 05/03. The default full cycle is:

Start time	Road 1	Road 2	Phase duration
0 s	Red 05	Green 03	3 s
3 s	Red 02	Amber 02	2 s
5 s	Green 03	Red 05	3 s
8 s	Amber 02	Red 02	2 s
10 s	Red 05	Green 03	Cycle repeats

For the video, demonstrate modes 02, 03 and 04 and confirm a new triplet 7/3/4: increase red twice, amber once and green once, pressing BTN2 in each mode. Return to normal and show the new countdowns. Then demonstrate that a held BTN1 increments only once, that unconfirmed edits are discarded, and that 99 wraps to 1. An optional invalid-triplet demonstration shows the error indicator and subsequent correction.

Verification

The project builds in STM32CubeIDE with zero errors and zero warnings. Automated tests check button debouncing, one increment per press, mode selection, saving and discarding edits, the 99-to-1 wrap, and recovery from an invalid timing configuration. They also check the countdown sequence and confirm that the two roads do not show green simultaneously in normal mode. Timer tests cover interrupt periods of 1, 10 and 100 ms, with the resolution limitation described in Exercise 4.

The HEX record checksums and firmware contents are checked before loading the program into Proteus. Circuit-level verification uses the mode sequence, displayed values and traffic-light timing listed above as the expected results.

Demo Lab 3